

APOSTILA

Lógica de Programação

Autor: Claude.ai

Conceitos · Exemplos · Exercícios

Conteúdo deste material

1. Algoritmos e Fluxogramas
2. Variáveis e Tipos de Dados
3. Operadores Aritméticos, Relacionais e Lógicos
4. Estruturas de Controle: Condicional e Repetição
5. Funções e Decomposição de Problemas
6. Complexidade de Algoritmos (Big-O)
7. Exercícios Práticos com Gabarito

1. Algoritmos e Fluxogramas

1.1 O que é um algoritmo?

Um algoritmo é uma sequência finita, ordenada e sem ambiguidade de passos que resolve um problema ou executa uma tarefa. Para ser válido, precisa ter início e fim definidos.

Definição formal: Algoritmo é um conjunto de instruções precisas e ordenadas que, quando executadas, produzem o resultado desejado para um determinado problema.

Características essenciais de um algoritmo

- Finitude — sempre termina após um número finito de passos
- Clareza — cada instrução é precisa e sem ambiguidade
- Entrada — recebe zero ou mais dados de entrada
- Saída — produz pelo menos um resultado
- Efetividade — cada passo é executável na prática

1.2 Exemplo do mundo real

Preparar um sanduíche é um algoritmo do cotidiano:

INÍCIO

1. Pegar duas fatias de pão
2. Passar manteiga em uma fatia
3. Colocar o recheio (queijo, presunto, etc.)
4. Fechar com a segunda fatia
5. Servir o sanduíche

FIM

Dica: Todo algoritmo tem ENTRADA (ingredientes), PROCESSAMENTO (os passos) e SAÍDA (o resultado). Essa estrutura vale para algoritmos computacionais também.

1.3 Fluxograma

Um fluxograma é a representação visual de um algoritmo usando símbolos padronizados. Serve para planejar e comunicar a lógica antes de escrever código.

Símbolo	Forma	Significado
Terminador	Oval	Início ou Fim do algoritmo
Processo	Retângulo	Instrução ou operação (ex: $x = x + 1$)
Decisão	Losango	Condição com dois caminhos (Sim / Não)
Entrada/Saída	Paralelogramo	Leitura de dados ou exibição de resultado
Conector	Seta	Indica a direção do fluxo de execução

1.4 Sequência para resolver um problema

A ordem correta ao desenvolver uma solução computacional é fundamental:

ETAPA 1 → Entender e definir o Problema
ETAPA 2 → Criar o Algoritmo (esboço lógico)
ETAPA 3 → Implementar o Código (programar)
ETAPA 4 → Testar e validar a solução

Atenção: Nunca pule direto para o código sem antes entender o problema e planejar o algoritmo. Esse é um dos erros mais comuns de iniciantes.

1.5 Exercícios — Algoritmos

Exercício 1

Descreva, em linguagem natural, um algoritmo com pelo menos 4 passos para realizar uma ação do dia a dia (ex: escovar os dentes, fazer café, atravessar a rua).

Dica: Lembre-se: deve ter início, passos ordenados e fim.

Resposta: _____

Exercício 2

Qual das opções abaixo representa corretamente um algoritmo? a) Ligar o computador b) Passos organizados para preparar um sanduíche c) Um arquivo de imagem d) Um banco de dados

Dica: Pense em qual opção descreve uma SEQUÊNCIA de passos.

Resposta: _____

Exercício 3

Coloque as etapas na ordem correta para resolver um problema computacional: ☐ Testar a solução ☐ Entender o problema ☐ Implementar o código ☐ Criar o algoritmo

Resposta: _____

Gabarito

Ex. 2: Letra b) — Apenas a opção b descreve uma sequência ordenada de passos.

Ex. 3: 2 → 4 → 3 → 1 — Problema → Algoritmo → Código → Teste.

2. Variáveis e Tipos de Dados

2.1 O que é uma variável?

Uma variável é um espaço nomeado na memória do computador usado para armazenar um dado que pode mudar durante a execução do programa. Pense como uma caixa com etiqueta: a etiqueta é o nome e o conteúdo é o valor.

```
x = 5           # x é uma variável que guarda o inteiro 5
nome = "Ana"    # nome guarda o texto Ana
ativo = True    # ativo guarda um valor lógico (booleano)
```

2.2 Tipos de dados

Tipo	Descrição	Exemplos
Inteiro (int)	Números sem casas decimais	5, -3, 100, 0
Real (float)	Números com casas decimais	3.14, -0.5, 2.0
Texto (string)	Sequência de caracteres entre aspas	"Olá", "ABC", "123"
Booleano (bool)	Valor lógico: verdadeiro ou falso	True, False

Importante: "123" com aspas é texto (string); 123 sem aspas é número inteiro. O tipo determina quais operações são possíveis com o dado.

2.3 Atribuição e reatribuição

O símbolo = atribui um valor a uma variável. Uma variável pode ser atualizada usando seu valor atual:

```
x = 5           # x recebe 5
x = x + 3       # x recebe seu valor atual (5) + 3 = 8
x = x * 2       # x recebe seu valor atual (8) * 2 = 16

# Rastreando linha por linha:
# Após linha 1: x = 5
# Após linha 2: x = 8
# Após linha 3: x = 16
```

Atenção: Em programação, = é ATRIBUIÇÃO (guardar um valor), não é igualdade matemática. Para comparar igualdade, usamos == (dois sinais de igual).

2.4 Exercícios — Variáveis

Exercício 4

Qual é o valor de x após a execução das linhas abaixo? `x = 3` `x = x * 2`

Dica: Substitua o valor de x no lado direito da segunda linha.

Resposta: _____

Exercício 5

Classifique cada dado abaixo no tipo correto (inteiro, real, string ou booleano): a) 42 b) "Computador" c) 3.14 d) False e) "99"

Resposta: _____

Exercício 6

O que o código abaixo exibe? `x = 10 y = 3 print(x * y)`

Dica: Calcule o resultado da operação.

Resposta: _____

Gabarito

Ex. 4: `x = 6` — `3 * 2 = 6`

Ex. 5: a) inteiro b) string c) real d) booleano e) string — "99" está entre aspas, logo é texto.

Ex. 6: 30 — `10 * 3 = 30`

3. Operadores

3.1 Operadores aritméticos

Operador	Operação	Exemplo	Resultado
+	Adição	7 + 3	10
-	Subtração	7 - 3	4
*	Multiplicação	7 * 3	21
/	Divisão real	7 / 2	3.5
%	Resto da divisão (módulo)	10 % 3	1
**	Potenciação	2 ** 3	8

O operador % (módulo) retorna o RESTO da divisão inteira. Por exemplo: $10 \% 3 = 1$, pois $10 \div 3 = 3$ com resto 1. É muito usado para verificar se um número é par ($n \% 2 == 0$).

3.2 Precedência (ordem de operações)

Assim como na matemática, os operadores têm prioridade diferente. A ordem é:

1. Parênteses: ()
2. Potenciação: **
3. Multiplicação/Divisão/Mod: * / %
4. Adição/Subtração: + -

Exemplos práticos de precedência:

$5 + 3 * 2 \rightarrow 5 + (3*2) = 5 + 6 = 11 \leftarrow \text{NÃO é } 16!$
 $3 + 4 * 2 \rightarrow 3 + (4*2) = 3 + 8 = 11$
 $x = 3$
 $x + 4 * 2 \rightarrow 3 + (4*2) = 3 + 8 = 11$
 $(5 + 3) * 2 \rightarrow 8 * 2 = 16 \leftarrow \text{com parênteses muda tudo}$

Regra de ouro: quando tiver dúvida sobre precedência, use parênteses. $(5 + 3) * 2$ é mais claro do que depender de saber a precedência de cabeça.

3.3 Operadores de comparação (relacionais)

Sempre retornam um valor booleano: True (verdadeiro) ou False (falso).

Operador	Significado	Exemplo	Resultado
==	Igual a	5 == 5	True
!=	Diferente de	5 != 3	True
>	Maior que	10 > 5	True

Operador	Significado	Exemplo	Resultado
<	Menor que	$3 < 2$	False
>=	Maior ou igual	$5 >= 5$	True
<=	Menor ou igual	$4 <= 3$	False

3.4 Operadores lógicos

Combinam expressões booleanas para formar condições mais complexas:

Operador	Significado	Exemplo	Resultado
AND	E — ambos devem ser True	True AND False	False
OR	OU — pelo menos um True	True OR False	True
NOT	NÃO — inverte o valor	NOT True	False

3.5 Exercícios — Operadores

Exercício 7

Calcule o resultado das expressões abaixo: a) $5 + 3 * 2$ b) $(5 + 3) * 2$ c) $10 \% 3$ d) $2 ** 4$ e) $3 + 4 * 2$

*Dica: Lembre-se da ordem de precedência: parênteses, depois *, depois +.*

Resposta: _____

Exercício 8

Qual o resultado lógico (True ou False) de cada expressão? a) $10 > 5$ b) $3 == 4$ c) $5 == 5$ d) $7 != 7$

Dica: Operadores de comparação sempre retornam True ou False.

Resposta: _____

Exercício 9

Dado $x = 3$, $y = 4$, qual o resultado de: `print(x + y * 2)`

*Dica: Cuidado com a precedência: $y * 2$ é calculado antes de $x + \dots$*

Resposta: _____

Gabarito

Ex. 7a: 11 — $3*2=6$, depois $5+6=11$

Ex. 7b: 16 — parênteses primeiro: $5+3=8$, depois $8*2=16$

Ex. 7c: 1 — $10 \div 3 = 3$ com resto 1

Ex. 7d: 16 — $2^4 = 2*2*2*2 = 16$

Ex. 7e: 11 — $4*2=8$, depois $3+8=11$

Ex. 8a: True — 10 é maior que 5

Ex. 8b: False — 3 não é igual a 4

Ex. 8c: `True` — 5 é igual a 5

Ex. 8d: `False` — 7 é igual a 7, portanto não é diferente

Ex. 9: 11 — $4*2=8$, depois $x+8 = 3+8=11$

4. Estruturas de Controle

4.1 Estrutura sequencial

Os comandos são executados um após o outro, na ordem em que aparecem no código. É a estrutura mais básica e está presente em todo algoritmo.

```
x = 10
y = 5
resultado = x + y
print(resultado) # Saída: 15
```

4.2 Estrutura condicional (se / senão)

Permite que o algoritmo tome decisões com base em uma condição. Se a condição for True, um bloco é executado; caso contrário, outro bloco é executado (ou nenhum).

```
# Sintaxe geral
se <condição>:
    # executado se condição for True
senão:
    # executado se condição for False

# Exemplo concreto
x = 10
se x > 5:          # 10 > 5 é True
    mostrar "A"    # → este bloco é executado
senão:
    mostrar "B"    # → este bloco é ignorado
# Saída: A
```

Outro exemplo — condição falsa:

```
x = 5
y = 10
se x > y:          # 5 > 10 é False
    mostrar "A"    # → ignorado
senão:
    mostrar "B"    # → este bloco é executado
# Saída: B
```

4.3 Estrutura de repetição — for (para)

Executa um bloco um número determinado de vezes. Use quando souber quantas iterações serão necessárias.

```
# Conta de 1 a 4 — executa 4 vezes
para i de 1 até 4:
    imprimir(i)

# Saída:
# 1
# 2
# 3
# 4
```

Para contar o número de iterações de 'para i de A até B', use a fórmula: $B - A + 1$.
Exemplo: de 1 até 4 $\rightarrow 4 - 1 + 1 = 4$ iterações.

4.4 Estrutura de repetição — while (enquanto)

Executa um bloco ENQUANTO uma condição for verdadeira. Use quando não souber antecipadamente quantas iterações serão necessárias.

```
i = 1
while i <= 3:
    imprimir(i)
    i = i + 1      # ESSENCIAL: atualizar a variável de controle

# Saída:
# 1
# 2
# 3

# Rastreando passo a passo:
# Início: i = 1. Condição: 1 <= 3? Sim → imprime 1, i vira 2
# Passo 2: i = 2. Condição: 2 <= 3? Sim → imprime 2, i vira 3
# Passo 3: i = 3. Condição: 3 <= 3? Sim → imprime 3, i vira 4
# Passo 4: i = 4. Condição: 4 <= 3? Não → para o loop
```

Cuidado com loop infinito! Se você esquecer de atualizar a variável de controle dentro do while, o programa nunca para. Sempre certifique-se de que a condição eventualmente se tornará False.

4.5 Rastreando código com loop de acumulação

Técnica essencial: execute o código mentalmente, linha por linha, atualizando os valores das variáveis em uma tabela:

```
x = 1
para i de 1 até 3:
    x = x + 2
```

Iteração (i)	Valor de x antes	Operação $x = x + 2$	Valor de x depois
i = 1	1	$x = 1 + 2$	3
i = 2	3	$x = 3 + 2$	5
i = 3	5	$x = 5 + 2$	7

Valor final de x: 7

4.6 Exercícios — Estruturas

Exercício 10

O que é exibido pelo código abaixo? $x = 10$ se $x > 5$: mostrar "A" senão: mostrar "B"

Dica: Avalie a condição $x > 5$ com $x = 10$.

Resposta: _____

Exercício 11

Quantas vezes o loop abaixo executa? O que é impresso? para i de 1 até 4: imprimir(i)

Resposta: _____

Exercício 12

Rastreie o código e informe o valor final de x: $x = 3$ $x = x * 2$

Dica: Substitua o valor atual de x na operação.

Resposta: _____

Exercício 13

Qual estrutura de repetição é mais adequada quando NÃO sabemos antecipadamente quantas vezes o bloco será executado?

Dica: Pense: for é para quando sabemos; qual é para quando não sabemos?

Resposta: _____

Exercício 14

Rastreie o código e informe o valor final de x: $x = 1$ para i de 1 até 3: $x = x + 2$ Use a tabela: i=1: x = ____ i=2: x = ____ i=3: x = ____ Valor final: x = ____

Resposta: _____

Gabarito

Ex. 10: "A" — $10 > 5$ é True, portanto o bloco do 'se' é executado.

Ex. 11: 4 vezes; imprime 1, 2, 3, 4 — de 1 até 4 são 4 iterações.

Ex. 12: $x = 6$ — $x = 3 * 2 = 6$

Ex. 13: while (enquanto) — O while verifica a condição a cada iteração, sem contador fixo.

Ex. 14: $x = 7$ — $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ (cada iteração soma 2)

5. Funções

5.1 O que é uma função?

Uma função é um bloco de código nomeado e reutilizável que realiza uma tarefa específica. Funções permitem dividir um problema grande em partes menores e independentes.

```
# Definindo uma função
def saudacao(nome):
    mensagem = "Olá, " + nome
    return mensagem

# Chamando (usando) a função
resultado = saudacao("Maria")
print(resultado) # Saída: Olá, Maria

resultado2 = saudacao("João")
print(resultado2) # Saída: Olá, João
```

5.2 Por que usar funções?

- Reutilização — escreva uma vez, use em vários lugares
- Organização — código dividido em blocos com propósito claro
- Manutenção — corrigir um erro em uma função resolve o problema em todos os usos
- Legibilidade — o nome da função descreve o que o código faz
- Decomposição — resolve problemas complexos dividindo-os em partes simples

Princípio importante: uma função bem projetada faz apenas UMA coisa e faz bem. Funções com muita responsabilidade ficam difíceis de entender e manter.

5.3 Exercícios — Funções

Exercício 15

Qual estrutura de programação é usada para dividir um problema em partes menores e reutilizáveis? a) variável b) função c) loop d) condicional

Resposta: _____

Exercício 16

Cite duas vantagens de usar funções em vez de repetir o código diretamente.

Resposta: _____

Gabarito

Ex. 15: Letra b) função — *Funções encapsulam lógica e permitem reutilização.*

Ex. 16: Reutilização de código + Facilidade de manutenção — *Entre outras: organização, legibilidade, decomposição.*

6. Complexidade de Algoritmos (Big-O)

6.1 O que é complexidade?

A complexidade mede como o tempo de execução de um algoritmo cresce conforme o tamanho da entrada (N) aumenta. Usamos a notação Big-O para expressar isso. Ela nos diz o quanto o algoritmo fica mais lento quando os dados crescem.

6.2 Os casos mais comuns

Notação	Nome	Quando ocorre	Exemplo
$O(1)$	Constante	Acesso direto, sem depender de N	Acessar lista[3]
$O(\log n)$	Logarítmica	Divide o problema pela metade a cada passo	Busca binária
$O(n)$	Linear	Um único loop que percorre N elementos	Busca simples
$O(n^2)$	Quadrática	Dois loops aninhados sobre N elementos	Bubble sort

6.3 Exemplos de código por complexidade

Complexidade $O(n)$ — um único loop percorrendo a lista:

```
# O(n) - percorre cada elemento uma vez
para cada item em lista:
    processar(item)
# Se lista tem 100 itens → 100 operações
# Se lista tem 1000 itens → 1000 operações
```

Complexidade $O(n^2)$ — dois loops aninhados:

```
# O(n²) - dois loops, cada um com N iterações
para cada item_A em lista:
    para cada item_B em lista:
        comparar(item_A, item_B)
# Se lista tem 100 itens → 100 × 100 = 10.000 operações
# Se lista tem 1000 itens → 1.000.000 de operações!
```

Impacto real: Com N = 1.000 elementos:

$O(n)$ executa 1.000 operações

$O(n^2)$ executa 1.000.000 operações

A diferença fica exponencialmente maior conforme N cresce.

6.4 Exercícios — Complexidade

Exercício 17

Qual é a complexidade de um algoritmo que percorre uma lista de N elementos UMA única vez?

Dica: Pense: quantas operações são feitas em relação a N?

Resposta: _____

Exercício 18

Um algoritmo tem dois loops aninhados, cada um percorrendo N elementos. Qual a complexidade?

Dica: Multiplique o número de iterações de cada loop.

Resposta: _____

Exercício 19

Ordene as complexidades do mais eficiente (mais rápido) para o menos eficiente: $O(n^2)$, $O(1)$, $O(n)$, $O(\log n)$

Resposta: _____

Gabarito

Ex. 17: $O(n)$ — Linear — Um loop simples percorrendo N elementos = N operações.

Ex. 18: $O(n^2)$ — Quadrática — $N \times N = N^2$ operações no total.

Ex. 19: $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n^2)$ — $O(1)$ é mais rápido; $O(n^2)$ é mais lento.

7. Exercícios de Rastreamento e Revisão Geral

Rastreamento de código — trace table

O rastreamento (ou trace) consiste em executar o código manualmente, anotando o valor de cada variável após cada linha. É a habilidade mais importante para entender o que um programa faz.

Como fazer um trace:

8. Leia o código linha por linha, da primeira à última
9. Para cada linha, atualize o valor de TODAS as variáveis afetadas
10. Para estruturas condicionais: avalie a condição e siga apenas o caminho correto
11. Para loops: repita os passos para cada iteração
12. O resultado final é o estado de todas as variáveis após a última linha

Exercício 20

Trace e informe a saída do código: `x = 5 y = 10 se x > y:    mostrar "A" senão:    mostrar "B"`

Dica: Avalie $5 > 10$. Qual é o resultado lógico?

Resposta: _____

Exercício 21

Trace e informe a saída do código: `x = 3 y = 4 print(x + y * 2)`

*Dica: Lembre-se da precedência: $y * 2$ é calculado primeiro.*

Resposta: _____

Exercício 22

Trace e informe a saída do código: `x = 10 y = 3 print(x % y)`

Dica: O operador % retorna o RESTO da divisão. $10 \div 3 = ?$

Resposta: _____

Exercício 23

Trace e informe a saída do código: `x = 2 y = 3 print(x ** y)`

*Dica: O operador ** é potenciação. x elevado a y .*

Resposta: _____

Exercício 24

Trace e informe a saída do código: `x = 5 y = 5 print(x == y)`

Dica: O operador == compara igualdade e retorna True ou False.

Resposta: _____

Exercício 25

Quantas vezes o loop while abaixo executa? `i = 1 while i <= 3: imprimir(i) i = i + 1`
 Faça o trace completo com tabela: `i=1: condição __, imprime __, i vira __ i=2: condição __, imprime __, i vira __ i=3: condição __, imprime __, i vira __ i=4: condição __ → para`
 Resposta: _____

Revisão — Tabela de operadores e estruturas

Símbolo/Estrutura	Tipo	Descrição
=	Atribuição	Guarda um valor em uma variável
==	Comparação	Verifica se dois valores são iguais → True/False
+ - * /	Aritmético	Operações matemáticas básicas
%	Aritmético	Resto da divisão (módulo)
**	Aritmético	Potenciação (x elevado a y)
> < >= <=	Relacional	Comparações → retornam True/False
AND OR NOT	Lógico	Combinam condições booleanas
se / senão	Condicional	Tomada de decisão no algoritmo
para (for)	Repetição	Loop com número fixo de iterações
enquanto (while)	Repetição	Loop com condição — executa até ser False
def / função	Função	Bloco reutilizável que resolve um subproblema

Gabarito

Ex. 20: "B" — `5 > 10` é False → executa o senão.

Ex. 21: 11 — `y*2=8, x+8 = 3+8 = 11`.

Ex. 22: 1 — `10 ÷ 3 = 3` com resto 1.

Ex. 23: 8 — `2³ = 8`.

Ex. 24: True — `5 == 5` é True.

Ex. 25: 3 vezes; imprime 1, 2, 3 — `i=1,2,3` satisfazem `i<=3`; `i=4` não satisfaz → para.